🚀 Neon Tracker - Task Management API

Una plataforma completa para la gestión de tareas ("objetivos") con una interfaz gráfica moderna (Neon UI) y una API RESTful robusta en el backend. El sistema permite a los usuarios registrarse, iniciar sesión y gestionar sus tareas de forma privada.

🛠 Tecnologías Usadas

El proyecto está construido bajo una arquitectura cliente-servidor completa y basada en la abstracción por APIs:

Backend

- **Node.js**: Entorno de ejecución principal y motor del proyecto.
- **Express.js**: Framework rápido y minimalista para inicializar el servidor y manejar las directivas HTTP.
- **SQLite (sqlite3)**: Base de datos ligera e integrada (en un único archivo local `database.sqlite`).
- **OpenSpec (OpenAPI 3.0)**: Todo el diseño, el comportamiento del backend y la estructura de los objetos está dictado e implementado a partir de nuestra especificación central en `api.yaml`.
- **JSON Web Token (JWT) y bcryptjs**: Herramientas integradas para garantizar el manejo de la autenticación mediante tokens y un encriptado robusto de todas las contraseñas guardadas en la base de datos.

Frontend

- **HTML, CSS & Vanilla JS**: Las páginas web en la carpeta `public/` están escritas de forma nativa.
- **Tailwind CSS**: Agregado vía CDN para construir el tema "Neon UI" (paneles oscuros con resaltado en naranja intenso).
- **Chart.js**: Uso interno para el renderizado del dashboard y las métricas animadas.

🛡 Seguridad Implementada

Proteger los datos contra intrusiones es una prioridad del sistema.

1. **Gestión Estricta de Roles (`admin` y `user`)**:
    - Para las rutas dedicadas (ej. `/usuarios` o `/dashboard`), el backend utiliza un *middleware verificador* (`verifyAdmin`) que comprueba obligatoriamente que el usuario de la petición posea el rol `'admin'`.
    - Los usuarios normales (`'user'`) solo pueden ver o editar **sus propias tareas** (gracias a validaciones adicionales sobre el `usuario_id`). Si un usuario estándar intenta borrar a alguien o mirar tareas ajenas, recibe una prohibición por código `403 Forbidden`.
2. **Prevención de Inyecciones SQL**:
    - En lugar de construir tablas de cadenas crudas SQL concatenadas (ej. `SELECT * FROM x WHERE id = ' + param`), se implementa de manera estricta el **uso de consultas con parámetros (`Parametrized Statements`)** en el paquete `sqlite3`.

- Todos los inputs y dinámicas del frontend (ej. `[username, password]`) se envían de forma higienizada como un arreglo de valores mediante el comodín `?` (`db.get('... WHERE username = ?', parametro)`). Esto detiene por absoluto cualquier ataque básico de inyección SQL.

⚙ Instrucciones de Ejecución y Pruebas

Para probar este proyecto en tu entorno local, sigue los pasos a continuación:

1. Preparar el Entorno

Asegúrate de que tienes `Node.js` instalado. Desde tu terminal en la raíz del proyecto, instala las dependencias necesarias:

```
npm install
```

2. Ejecutar la Aplicación

Arranca el programa levantando el servidor express:

```
node server.js
```

(El servidor mostrará un mensaje indicando el puerto de inicio, usualmente el puerto `3000`).

3. Prueba Gráfica (Login y Dashboards)

Abre tu navegador web e ingresa en: **http://localhost:3000**

Podrás registrar una "Nueva identidad" en el menú, o alternativamente puedes iniciar sesión con las cuentas de demostración del sistema pre-generadas en la base de datos:

Acceso Administrador

- *Usuario:* `admin`
- *Contraseña:* `admin123` *(Acceso completo a Dashboard Global y lista de Usuarios)*

Acceso Usuario Estándar

- *Usuario:* `demo`
- *Contraseña:* `demo123` *(Visión restringida, solo Dashboard Personal y control propio)*